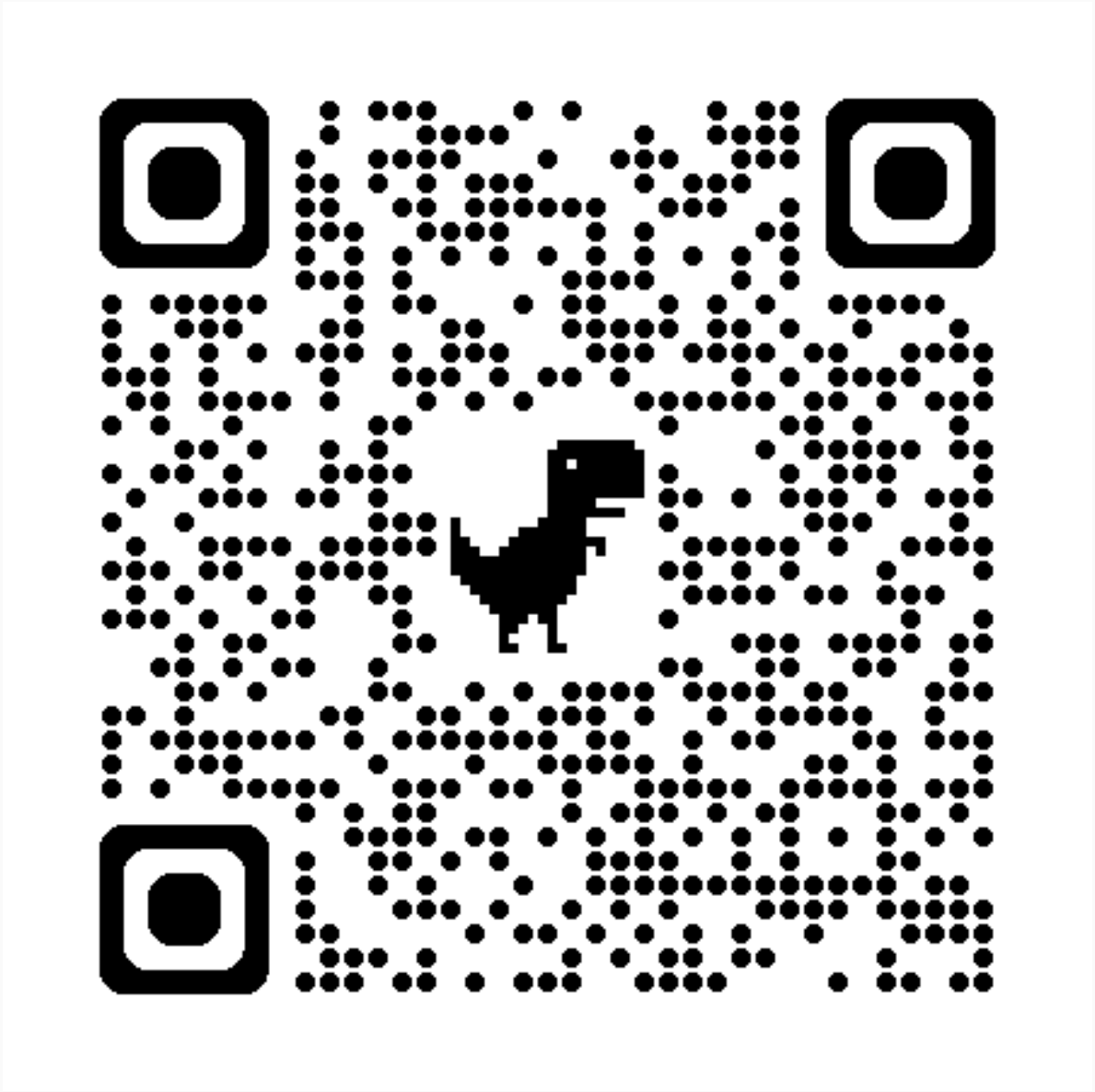
Binary Search

Tai, Wei Hsuan

week 3

課程簡報



Outline

1. Introduction

2. Basic Binary Search

3. Range Binary Search

Introduction

What is Binary Search

Let's play a game. I will think of a number between 1 and 100, and you have to guess it. After each guess, I will tell you whether your guess is too high, too low, or correct. How many guesses do you need to find the number?

Basic Concept

The main idea of binary search is ”split into half”. When we guess a number, we can eliminate half of the remaining possibilities based on whether our guess is too high or too low. This way, we can quickly narrow down our search and find the correct number in a logarithmic number of guesses.

Logarithm

We know that a^n represents the number a multiplied by itself n times. For example, $2^3 = 2 \times 2 \times 2 = 8$. But what if we want to find out how many times we need to multiply a by itself to get a certain number x? This is where logarithms come in.

The logarithm of a number x to the base a is denoted as $\log_a(x)$ and is defined as the exponent n such that $a^n = x$.

In computer science, logarithmic time complexity is a strong indicator. For instance, $\log(10^5) \approx 15$, $\log(10^9) \approx 27$

Basic Binary Search

Compositions of Binary Search

- Main loop
- Boundary control
- Termination conditions

Procedure of Binary Search

1. Set the initial search range
2. Check the middle element of the current search range
3. If the middle element is the target, return its index
4. If the middle element is less than the target, update the search range to the right half
5. If the middle element is greater than the target, update the search range to the left half
6. Repeat steps 2-5 until the search range is empty

Example

Find 4 in the array [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

Code Implementation

```
1  int bs(int l, int r, int target){
2      while(true){
3          int mid=(l+r)/2;
4          if(arr[mid]==target) return mid;
5          if(l==r)return -1;
6
7          if(arr[mid]>target) r=mid;
8          else l=mid+1;
9      }
10     return -1;
11 }
```

Example

Given an array with N elements $A_1 \sim A_N$ and a target value T. Find a range $[L, R]$ such that $\sum_{i=L}^{R} A_i = T$.

Solution 1: Plain Solution

Enumerate all possible ranges and check if the sum equals T. We have to enumerate $O(N^2)$ ranges, and each range takes $O(N)$ time to calculate the sum, resulting in a total time complexity of $O(N^3)$.

Solution 2: Prefix Sum

We can use prefix sums to optimize the calculation of subarray sums. By precomputing the prefix sums, we can calculate the sum of any subarray in constant time. This reduces the overall time complexity to $O(N^2)$ for enumerating all possible ranges.

Solution 3: Binary Search + Prefix Sum

We can further optimize the solution by using binary search to find the right endpoint of the range. For each left endpoint L, we can use binary search to find the smallest right endpoint R such that the sum from L to R is at least T. This way, we can reduce the time complexity to $O(N \log N)$.

Range Binary Search

Sometimes, the problem is to complex to solve. But it can verify the solution in a sufficient time. In this case, we can enumerate the answer and check if it is correct. This is called range search.
However, the range may be too large (e.g. 10^9), so we can use binary search to find the answer.

Example: Wood Cutting

Given N woods $A_1 \sim A_N$ and a number k, we want to cut the woods into pieces of length x such that we can get at least k pieces. Find the maximum value of x. Note that each wood should be cut into the same length.

Wood Cutting

It is hard to find the answer directly, but we can check if a given length x is valid by counting how many pieces we can get from the woods. If we can get at least k pieces, then x is a valid length. We can use binary search to find the maximum valid length.